



Fruits into Baskets (medium)

We'll cover the following ^

- Problem Statement
- Try it yourself
- Solution
- Code
 - Time Complexity
 - Space Complexity
- Similar Problems

Problem Statement

You are visiting a farm to collect fruits. The farm has a single row of fruit trees. You will be given two baskets, and your goal is to pick as many fruits as possible to be placed in the given baskets.

You will be given an array of characters where each character represents a fruit tree. The farm has following restrictions:

1. Each basket can have only one type of fruit. There is no limit to how many fruit a basket can hold.
2. You can start with any tree, but you can't skip a tree once you have started.
3. You will pick exactly one fruit from every tree until you cannot, i.e., you will stop when you have to pick from a third fruit type.

Write a function to return the maximum number of fruits in both baskets.

Example 1:

Input: Fruit=['A', 'B', 'C', 'A', 'C']

Output: 3

Explanation: We can put 2 'C' in one basket and one 'A' in the other from the subarray ['C', 'A', 'C']

Example 2:

Input: Fruit=['A', 'B', 'C', 'B', 'B', 'C']

Output: 5

Explanation: We can put 3 'B' in one basket and two 'C' in the other basket.

This can be done if we start with the second letter: ['B', 'C', 'B', 'B', 'C']



Try it yourself

Try solving this question here:





```

1  import java.util.*;
2
3  class MaxFruitCountOf2Types {
4      public static int findLength(char[] a) {
5          int s=0, max = 0;
6          Map<Character, Integer> m = new HashMap<>();
7          for(int e=0; e < a.length; e++){
8              m.put(a[e], m.getOrDefault(a[e], 0) + 1);
9              if(m.size() > 2){
10                 m.put(a[s], m.get(a[s]) - 1);
11                 if(m.get(a[s]) == 0)
12                     m.remove(a[s]);
13                 s++;
14             }
15             max = Math.max(max, e-s+1);
16         }
17         return max;
18     }
19 }
20

```

Test

Save

Reset



Solution

This problem follows the **Sliding Window** pattern and is quite similar to [Longest Substring with K Distinct Characters](#). In this problem, we need to find the length of the longest subarray with no more than two distinct characters (or fruit types!). This transforms the current problem into **Longest Substring with K Distinct Characters** where $K=2$.

Code

Here is what our algorithm will look like, only the highlighted lines are different from [Longest Substring with K Distinct Characters](#):



```

1  import java.util.*;
2
3  class MaxFruitCountOf2Types {
4      public static int findLength(char[] arr) {
5          int windowStart = 0, maxLength = 0;
6          Map<Character, Integer> fruitFrequencyMap = new HashMap<>();
7          // try to extend the range [windowStart, windowEnd]
8          for (int windowEnd = 0; windowEnd < arr.length; windowEnd++) {
9              fruitFrequencyMap.put(arr[windowEnd], fruitFrequencyMap.getOrDefault(arr[windowEnd], 0) + 1);
10             // shrink the sliding window, until we are left with '2' fruits in the frequency map
11             while (fruitFrequencyMap.size() > 2) {
12                 fruitFrequencyMap.put(arr[windowStart], fruitFrequencyMap.get(arr[windowStart]) - 1);
13                 if (fruitFrequencyMap.get(arr[windowStart]) == 0) {
14                     fruitFrequencyMap.remove(arr[windowStart]);
15                 }
16                 windowStart++; // shrink the window
17             }
18         }
19         return maxLength;
20     }
21 }
22

```

```
17         }
18         maxLength = Math.max(maxLength, windowEnd - windowStart + 1);
19     }
20
21     return maxLength;
22 }
23
24 public static void main(String[] args) {
25     System.out.println("Maximum number of fruits: " +
26         MaxFruitCountOf2Types.findLength(new char[] { 'A', 'B', 'C', 'A', 'C' }));
27     System.out.println("Maximum number of fruits: " +
28         MaxFruitCountOf2Types.findLength(new char[] { 'A', 'B', 'C', 'B', 'B', 'C' }));
29 }
30 }
31
```

Run

Save

Reset



Time Complexity

The above algorithm's time complexity will be $O(N)$, where 'N' is the number of characters in the input array. The outer **for** loop runs for all characters, and the inner **while** loop processes each character only once; therefore, the time complexity of the algorithm will be $O(N + N)$, which is asymptotically equivalent to $O(N)$.

Space Complexity

The algorithm runs in constant space $O(1)$ as there can be a maximum of three types of fruits stored in the frequency map.

Similar Problems

Problem 1: Longest Substring with at most 2 distinct characters

Given a string, find the length of the longest substring in it with at most two distinct characters.

Solution: This problem is exactly similar to our parent problem.

[← Back](#)[✓ Completed](#)[Next →](#)

Longest Substring with maximum K Distinct Characte...

Longest Substring with Distinct Characters (hard)